

Mastering Helm Values and Overrides for Beginners

Helm values and overrides let you reuse the exact same application template across development, staging, and production environments. Learning this skill saves you from writing duplicate configuration files and keeps your Kubernetes deployments clean.

What you'll learn:

1. Understanding values.yaml
2. Using Custom Values Files for Environments
3. Quick Overrides with the --set Flag
4. Previewing Changes with Dry Run

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Understanding values.yaml

Every Helm chart comes with a default values.yaml file that contains configurable settings like image tags, ports, and resource limits. Think of it as a configuration menu where you define standard defaults for your application. When you install a chart without extra flags, Helm automatically pulls values straight from this file.

```
$replicaCount: 1
image:
  repository: nginx
  tag: latest
service:
  port: 80
```

Cloud & DevOps Daily

Helm Values & Overrides: Deploy the Same Chart Everywhere

Thursday, October 01, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Using Custom Values Files for Environments

Different environments need different settings, such as higher replica counts for production. Instead of editing the main template, you create a separate file like `values-prod.yaml` containing only the settings you want to change. Helm will merge your custom file on top of the default values during installation.

```
$helm install my-web-app ./my-chart -f values-prod.yaml
```

Cloud & DevOps Daily

Helm Values & Overrides: Deploy the Same Chart Everywhere

Thursday, October 01, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Quick Overrides with the --set Flag

When you need to change a single value quickly without making a new file, the `--set` flag is your best friend. It lets you pass parameters directly inside your terminal command. This is super handy for quick testing or injecting dynamic tags during a CI/CD pipeline run.

```
$helm install my-web-app ./my-chart --set image.tag=v1.2.0 --set replicaCount=3
```

Cloud & DevOps Daily

Helm Values & Overrides: Deploy the Same Chart Everywhere

Thursday, October 01, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Previewing Changes with Dry Run

Before applying your values files to a live cluster, always test them locally using the dry-run flag. This tells Helm to generate all Kubernetes manifests and print them to your screen without actually creating anything. It is the safest way to catch syntax errors and missing values early.

```
$helm install my-web-app ./my-chart -f values-prod.yaml --dry-run --debug
```

Cloud & DevOps Daily

Helm Values & Overrides: Deploy the Same Chart Everywhere

Thursday, October 01, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Keep environment-specific overrides in separate files like values-staging.yaml and values-prod.yaml.
- + Use helm template or --dry-run to verify your merged values before deploying.
- + Document custom parameters at the top of your custom values files for team visibility.
- + Store sensitive passwords and API keys in secret managers rather than plain values files.
- + Use descriptive key names in your templates so other engineers understand what overrides do.

Watch Out For

- ! Hardcoding environment values directly inside the default values.yaml file instead of overriding them.
- ! Forgetting that array overrides in Helm completely replace existing lists instead of merging them.
- ! Putting raw database passwords or API keys directly into public values files.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh